

ECE 2036 Lab 1: COVID-19 Calculator in C/C++

“A Global Variable Problem”

Category: Basic C/C++ Constructs and Software Engineering Principle “Incremental Design and Debug Philosophy”

Assigned: August 24, 2020

Due: September 2, 2020 at 11:59 PM

Topics Needed to Complete Lab: Topic 1, 2, and some of 4

Creating software systems to help us understand current problems and issues is important in your development as a software engineer. This assignment will allow you to explore managing a set of equations with many inputs in the context of a model that explores the risk of contracting COVID-19 in a classroom. Please note that I am not an expert in the field of epidemiology, and I am not necessarily endorsing this model. This model has been highlighted in various media outlets and was created at the University of Colorado by Dr. Jose L. Jimenez. You can find out more about his model at:

<https://docs.google.com/spreadsheets/u/0/d/16K1OQkLD4BjgBdO8ePj6ytf-RpPMIJ6aXFg3PrIQBbQ/htmlview#>

In this assignment you will make a program that will estimate the viral transmission rate in a closed room of a certain volume. My goal for this assignment is that I want you to consider the issues of creating a small software system that has a variety of input parameters. I want you to consider some basic ways to manage the data, calculate the equations, and display the results. A key concept in software engineering is debugging and minimizing errors in the code, and I would like you to consider this in this lab.

I have provided in Appendix A and B a set of equations and input parameters that are needed for Dr. Jimenez’s model that you can change to estimate the risk of contracting COVID-19 that you might have in a classroom. On Canvas I have also provided an input file that contains all the parameters in a file called `parameters.txt`. The goal of this assignment is for you to explore some basic constructs in C/C++. Your next lab will be using C++ classes, but in this lab you will explore the use of the following:

1. Basic file I/O manipulation

You are required to read in the input parameters from a text file `parameters.txt` and use these parameters to calculate the appropriate outputs given in the `output.dat` file that is provided to you. Please make sure that your data (call your file `my_output.dat`) matches the `output.dat` file that I provide to you exactly. I also have you create a file called `mask_data.dat` that shows the probability of infections as a function of the

number of people wearing masks in the classroom. I would encourage you to make a plot of this file using Excel. The CSV format should facilitate this.

2. Basic C/C++ mathematical manipulation

You will need to code the formulas in Appendix A as global functions. You will need the exponential function and power function that is part of the C math library. Please note the inclusion of `#include <cmath>` so that you can use the C math library. I have provided some documentation of this in Appendix A.

3. Basic global function manipulation and global function prototypes

Global functions can be called anywhere in the file. For this lab I will have you put all the global functions in the `main.cc` file.

4. Basic global variable definition and manipulation

The use of global variables is not good programming practice, but we will embrace it in this lab so that you understand their use. I have initialized the global variables for you in the skeleton code that I provide for you. You will need to read in their value from the input file `parameters.txt`.

5. Using functional programming to deal with complex modeling

Our next lab will use an object-oriented programming approach, but I wanted to start with this example for contrast.

Organization and Debugging

The first question you have to deal with is managing complexity. My first general response to this is to organize yourself and your system in a clear way. In the book they use a “unified modeling language” to help organize your code. Before we dive into this, I would recommend general organization of your code with an understanding of the language you are going to write the program in.

For example, this first lab represents a functional programming approach and uses global functions in C/C++ to try to solve this problem. Therefore, it is very important to know what your inputs are, what your outputs are, what functions you need, and potentially being clear about the need for nested functions. The output.dat file and Appendix A shows the required outputs for this assignment. The following chart gives the interaction between the global functions that you will create.

<u>Input Parameters</u>	<u>First Level Functions</u> (dependent only on input parameters)	<u>Second Level Functions</u> (dependent on other functions and input parameters)
lengthOfRoomft widthOfRoomft heightOfRoomft durationOfEventInMins numRepsOfEvent ventilationWithOutsideAir decayRateOfVirus depositionToSurfaces additionalControlMeasures numPeoplePresent numInfectedPeopleInRoom fractionOfPopulationImmune breathingRateSusceptiblePeople quantaExhalationRate_Infected exhalationMaskEfficiency fractionPeopleWearingMasks inhalationMaskEfficiency infectionRateInYourArea	volOfRoom totalFirstOrderViralLossRate numPeopleSusceptible areaPerPersonInRoom personsPerSquareMeter netEmissionRateFromInfected	avgQuantaConcentrations quantaInhaledPerPerson probabilityOfInfection_Conditional avgNumCOVID_Cases_Conditional numInfectedPeopleatBeginningInClassroom_RegionalRates probabilityOfInfection_OneEvent_RegionalRates numInfectedPeopleAfterEvent_RegionalRate probabilityOfInfectionMultipleEvents_RegionalRate NumberOfCOVID_CasesOverTime_RegionalRate ventRatePerPerson volumePerPerson

Incremental Design and Debug Philosophy

Once you have this organization in the above chart, then this obviously suggests a debugging and verification strategy. This strategy will help to guide the evolution of your programming so that you can minimize errors in your final program. This IS NOT a write-source-code-as-quick-as-possible approach. This IS a coding methodology that helps make sure you don’t have hidden errors that can be very difficult to find if you put the code together all at once! Unfortunately, this is just like Morpheus telling Neo about the Matrix. Just as Neo has to experience the Matrix, students many times have to hit a wall and experience this issue before they are willing to embrace this “Incremental Design and Debug Philosophy.”

Here is what I would suggest for this programming assignment.

Code Version 1: Verify the reading in of all parameters.

Code Version 2: Create all first level functions and verify their operation.

Code Version 3: Create all secondary level functions and verify their operation.

Code Version 4: Create output file for test case to verify operation.

Self-Documenting Code

This is huge, but again tedious. I would encourage you to add comments to your code, but more importantly have what folks call “self-documenting code.” Give your variables meaningful names instead of x1, y1, b32, etc... Give your functions meaningful names as well. This way as one traces through the code it becomes obvious what the algorithm is.

Appendix A – Dr. Jimenez’s Model Equations

Here are the items that you will need to calculate and use in this assignment. Please note that you need to have a meaningful naming convention. I would like for you to make all inputs that you read into variables and I would like all the calculated values to be return values from global functions.

Please make sure you think about task decomposition and program evolution to create a highly robust system. I would like for you to use camel case for your variable or function names.

Note that you will need to include the cmath header file because you will need to use the following two mathematical functions:

$$e^x \Rightarrow \exp(x)$$

and

$$x^y \Rightarrow \text{pow}(x,y)$$

Note that these will return a float value if you pass a float value to the function.

Here are the calculations that you will need to create C/C++ global functions.

1. Volume of the room (m³)

volOfRoom

Classrooms are not always rectangular boxes. For the case for this lab, you can start with the dimensions given to you to estimate this as a rectangular box.

$$= \text{lengthOfRoomft} * \text{widthOfRoomft} * \text{heightOfRoomft} * (\text{conversionFactor})$$

2. Total first order loss rate of the infectious viral doses in the room (per hour).

totalFirstOrderViralLossRate

$$= \text{ventilationWithOutsideAir} + \text{decayRateOfVirus} + \text{depositionToSurfaces} + \text{additionalControlMeasures}$$

3. Number of people susceptible in your classroom

This is simply the number people in the room minus the number infected with some correction based on fraction of people in the population that are immune. Some individuals could already be immune for various reasons.

$$\begin{aligned} \text{numPeopleSusceptible} \\ = (\text{numPeoplePresent} - \text{numInfectedPeopleInRoom}) (1 - \text{fractionOfPopulationImmune}) \end{aligned}$$

4. Ventilation rate per person (L/s/person)

$$\begin{aligned} \text{ventRatePerPerson} \\ = (10/36) * \text{volOfRoom} * (\text{ventilationWithOutsideAir} + \text{additionalControlMeasures}) / (\text{numPeoplePresent}) \end{aligned}$$

5. Area per person in the room (square ft per person)

$$\begin{aligned} \text{areaPerPersonInRoom} \\ = \text{lengthOfRoomft} * \text{widthOfRoomft} / (\text{numPeoplePresent}) \end{aligned}$$

6. People per area in the room (persons per square meter)

$$\begin{aligned} \text{personsPerSquareMeter} \\ = \text{numPeoplePresent} / (\text{lengthOfRoomft} * \text{widthOfRoomft} * (0.305)^2) \end{aligned}$$

7. Volumetric Density in room (volume/person). Volume is in cubic meter.

$$\begin{aligned} \text{volumePerPerson} \\ = \text{volOfRoom} / \text{numPeoplePresent} \end{aligned}$$

8. Net emission Rate of Infectious Quanta (quanta/hour)

$$\begin{aligned} \text{netEmissionRateFromInfected} \\ = \text{quantaExhalationRate_Infected} * (1 - \text{exhalationMaskEfficiency} * \text{fractionOfPeopleWearingMasks}) * \text{numInfectedPeopleInRoom} \end{aligned}$$

The units on this are infectious doses (i.e. quanta) per hour.

9. Average Quanta Concentration (quanta/liter)

avgQuantaConcentration

$$= netEmissionRateFromInfected / (totalFirstOrderViralLossRate * volOfRoom) \\ * (1.0 - (1.0 / (totalFirstOrderViralLossRate * durationOfEventInMins / 60.0f))) \\ * (1.0 - \text{EXP}(-totalFirstOrderViralLossRate * durationOfEventInMins / 60.0f)))$$

10. Quanta inhaled per person

quantaInhaledPerPerson

$$= avgQuantaConcentration * breathingRateSusceptiblePeople * (durationOfEventInMins / 60.0f) * (1.0 - inhalationMaskEfficiency * fractionPeopleWearingMasks)$$

11. Conditional probability given the number of people at event (default 1)

This assumes that a condition is set for this calculation. The condition is that there is a set number of infected people in the classroom. For this example the input file sets this to 1.

probabilityOfInfection_Conditional

$$= 1 - \exp(-quantaInhaledPerPerson)$$

avgNumCOVID_Cases_Conditional

$$= numPeopleSusceptible * probabilityOfInfection_Conditional$$

12. Probability Based on Infection Rates in Your Area

numInfectedPeopleatBeginningInClassroom_RegionalRates

This is the number of people coming into the room as an initial condition.

$$= (numInfectedPeopleInRoom + numPeopleSusceptible) * infectionRateInYourArea$$

probabilityOfInfection_OneEvent_RegionalRates

$$= 1.0 - (1.0 - probabilityOfInfection_Conditional * infectionRateInYourArea)^{numPeopleSusceptible}$$

numInfectedPeopleAfterEvent_RegionalRate

This is the number of people that will be infected on average.

$$= (numInfectedPeoplePresentInRoom + numPeopleSusceptible) * probabilityOfInfection_OneEvent_RegionalRates$$

13. Probability of Infection Based on Repeated Events

(using probability of infection rate in the region)

probabilityOfInfectionMultipleEvents_RegionalRates

$= 1.0 - (1.0 - \text{probabilityOfInfection_OneEvent_RegionalRates})^{(\text{numRepsOfEvent})}$

numberOfCOVID_CasesOverTime_RegionalRate

$= \text{probabilityOfInfectionMultipleEvents_RegionalRates} * (\text{numPeopleSusceptible} - \text{numInfectedPeopleatBeginningInClassroom_RegionalRate})$

Appendix B – Input Parameters and Output File Results

This is given to you in the input file called `parameters.txt`. Please note that I also have an example of the output file that you will need to create with your program. This output file is called `output.dat`. I would like for you to create an output file called `my_output.dat`.

`parameters.txt`

```
25    lengthOfRoomft
20    widthOfRoomft
10    heightOfRoomft
50    durationOfEventInMins
30    numRepsOfEvent
3     ventilationWithOutsideAir
0.62  decayRateOfVirus
0.3   depositionToSurfaces
0     additionalControlMeasures
35    numPeoplePresent
1     numInfectedPeopleInRoom
0     fractionOfPopulationImmune
0.516 breathingRateSusceptiblePeople
25    quantaExhalationRate_Infected
.50   exhalationMaskEfficiency
1     fractionPeopleWearingMasks
.30   inhalationMaskEfficiency
0.01  infectionRateInYourArea
```

`output.dat`

```
The volume of your room [cubic meters]:          141.86
The total viral loss rate [per hour]:             3.92
The number of people susceptible in the room:      34
The square-ft area per person [ft^2/person]:      14.286
The persons per square meter [person/m^2]:         0.75249
The cubic meter volume per person is [m^3/person]: 4.0532
The ventilation rate per person in the room [L/s/person]: 3.3777
The quanta exhalation rate from infected people [quanta/hr]: 12.5
The average viral quanta concentration in the room [quanta/L]: 0.015859
The average quanta inhaled per person in the room [quanta]: 0.0047736

== CONDITIONAL CASE ==
The probability of infection IF 1 infected person(s) in the room [%]: 0.47622
The average number of new COVID cases for conditional case: 0.16192

== REGIONAL CASE ONE EVENT ==
Starting number of infected people based on regional infection rates in the room: 0.35
The probability of being infected given the regional rates [%]: 0.16179
The average number of infected people after a single event: 0.056626

== REGIONAL CASE MULTIPLE EVENTS ==
The probability of being infected after going to 30 [%]: 4.7415
The number of people expected to get COVID after 30 events: 1.5955
```

APPENDIX C - Creating Lab1 Program on PACE-ICE and Submission Instructions

Creating Source Code. Once logged into PACE-ICE, you will need to use one of several available text editors found on the PACE-ICE cluster. Those are vi/vim, nano and emacs. I will use vi in class, and I will recommend that you become familiar with this editor. There are many online tutorials to help you get used to vi. You could get started here.

<https://coderwall.com/p/adv71w/basic-vim-commands-for-getting-started>

Basic Unix Commands. Please make a directory (i.e. “folder”) called Lab1 where you keep your files. To make this directory use the following command in your home directory:

```
mkdir Lab1
```

To go into this directory from your home directory, you can use the following command:

```
cd Lab1
```

To get back to your home directory you can use the command:

```
cd
```

Compiling Source Code. On the PACE-ICE system we will be using the gnu g++ or gcc compiler. Use the following command to compile your source code and create an executable file called testProgram.

```
g++ main.cc -o testProgram
```

To run your program at the command prompt, type in the executable file name

```
./testProgram
```

Submitting Assignments. You will need to submit your entire directory Lab1 for this lab. I will have you create what is called a ‘tar archive’, i.e. a “tarball,” of the directory and this is what you will submit on Canvas. This way you will just submit one file

1. I would like for you to make a directory on PACE-ICE called Lab1. To make this directory, type the following while you are in your home directory.

```
mkdir Lab1
```

2. Please call your source file main.cc. You can call your output file my_output.dat. Your input file will be called parameter.dat. All of these files should be contained in your Lab1 directory. To get into this folder you can use at the command prompt:

```
cd Lab1
```

3. Please go back to your home directory. You can do this by typing cd at the command prompt.

4. At the home directory, please type the following command to create a compressed tarball of your work

```
tar -cvzf yourusernameLab1.tar.gz ./Lab1
```

5. As I showed you in class you will need to download this file to your local machine. You will then submit this yourusernameLab1.tar.gz to Canvas so that the TAs can grade it.

APPENDIX D - C/C++ Basic Syntax

You can look these up in the textbook for more information. We will talk in class in more depth, but I would like for you to experiment with these programming constructs.

A. if statements in C/C++

```
if (condition)
{
    //body of the if statement
}
```

B. if-else statements in C/C++

```
if (condition)
{
    //body of the if statement
}
else
{
    //body of the if statement
}
```

C. For loop example

```
int i;
for (i = 0; i <= upperLimit; i++) //i++ is called the increment operator that adds 1 to the value of i
{
    //body of for loop
}
```

D. While loop example

```
while (condition)
{
    //body of while loop
}
```

Appendix E: Good Programming Practices

Indentation

When using *if/for/while* statements, make sure you indent 2 to 4 spaces for the content inside those. For example ...

```
for(int i; i < 10; i++)  
    j = j + i;
```

If you have nested statements, you should use multiple indentions. Your *if/for/while* statement brackets `{ }` can follow two possible conventions. Each both getting their own line (like the *for* loop) OR the open bracket on the same line as the statement (like for the *if/else* statement) and closing bracket its own line. If you have *else* or *else if* statements after your *if* statement, they should be on their own line.

```
for(int i; i < 10; i++)  
{  
    if (i < 5) {  
        counter++;  
        k -= i;  
    }  
    else {  
        k += i;  
    }  
    j += i;  
}
```

Camel Case

This naming convention has the first letter of the variable be lower case, and the first letter in each new word be capitalized (e.g. `firstSecondThird`). This applies for functions and member functions as well! The main exception to this is class names, where the first letter should also be capitalized.

Variable and Function Names

Your variable and function names should be clear about what that variable or function is. Do not use one letter variables, but use abbreviations when it is appropriate (for example: “`imag`” instead of “`imaginary`”). The more descriptive your variable and function names are, the more readable your code will be. This is the idea behind self-documenting code.

Clear Comments

Some good opportunities to use comments are...

- Introducing a member function or class
- Introducing a section of code with long implementation
- Your name, class information, etc. at the beginning of the file

APPENDIX F: ECE 2036 Lab Grading Rubric

If a student's program runs correctly and produces the desired output, the student has the potential to get a 100 on his or her lab; however, TA's will **randomly** look through this set of "perfect-output" programs to look for other elements of meeting the lab requirements. The table below shows typical deductions that could occur.

In addition, if a student's code does not compile on `coc-ice.pace.gatech.edu`, then he or she will have an automatic 30% deduction on the lab. Code that compiles but does not match the sample output can incur a deduction from 10% to 30% depending on how poorly the output matches the output specified by the lab. This is in addition to the other deductions listed below or due to the student not attempting the entire assignment.

AUTOMATIC GRADING POINT DEDUCTIONS

Element	Percentage Deduction	Details
Does Not Compile on PACE-ICE System	30%	Program does not compile on pace ice cluster!
Does Not Match Output	10%-30%	The program compiles but doesn't match output. I would like for you to match the output exactly so that a diff command can be used.

ADDITIONAL GRADING POINT DEDUCTIONS FOR RANDOMLY SELECTED PROGRAMS

Element	Percentage Deduction	Details
Does not turn in with a tarball	10%	Does not use correct encapsulation in object-oriented objects
Does not demonstrate knowledge of global functions and global variables	10%	The TA's will look at how you use these in your code to make this determination.
Clear Self-Documenting Coding Styles	5%-15%	This can include incorrect indentation, using unclear variable names, unclear comments, or compiling with warnings. (See Appendix E)

LATE POLICY

Element	Percentage Deduction	Details
Late Deduction Function	score - $(20/24)*H$	H = number of hours (ceiling function) passed deadline note : Sat/Sun count has one day; therefore $H = 0.5*H_{weekend}$

Appendix G – Quanta Emission Rates

This is only provided to you if you would like to play around with this model further.

Resting – Oral breathing = 2.0 quanta/h

Resting – Speaking = 9.4 quanta/h

Resting – Loudly speaking = 60.5 quanta/h

Standing – Oral breathing = 2.3 quanta/h

Standing – Speaking = 11.4 quanta/h

Standing – Loudly speaking = 65.1 quanta/h

Light exercise – Oral breathing = 5.6 quanta/h

Light exercise – Speaking = 26.3 quanta/h

Light exercise – Loudly speaking = 170 quanta/h

Heavy exercise – Oral breathing = 13.5 quanta/h

Heavy exercise – Speaking = 63.1 quanta/h

Heavy exercise – Loudly speaking = 408 quanta/h